MASTER DEVELOPMENT BLUEPRINT

PART-3: DATABASE & DEVELOPMENT STANDARDS

School ERP

PostgreSQL | Spring Data JPA + Hibernate | Redis

Prepared by: Solution Architecture / Engineering Leadership Team

Document Version: 1.0

Date: 29 July 2026

Preamble

This document is Part-3 of the Master Development Blueprint (MDB) v1.0 for the School ERP System, continuing directly from Part-1: Project Foundation and Part-2: Security & API Standards. It defines the mandatory database architecture, persistence, transaction, caching, and development standards, using PostgreSQL, Spring Data JPA, Hibernate, Java 21, Spring Boot 3, and Redis as fixed by prior Parts — now applied specifically to the logical data model already established in Appendix-H and the Repository/Service component design already established in Appendix-J.

This document does not regenerate RGD v2.0, any prior Appendix, any UI/UX Screen Specification, or MDB Parts 1-2 — every reference below is by ID/name to those sources. It contains no source code, no SQL, no ER diagram, no database table definitions, no entity/repository/service implementation, no cache configuration, no migration scripts, and no examples — every standard is stated as a rule or convention only.

21. Database Architecture Standards

Fixes PostgreSQL as the concrete realization of the logical database architecture already defined in Appendix-H and the deployment topology already defined in Appendix-L Section 9-10.

- A single PostgreSQL schema hosts all 39 entities defined in Appendix-H, organized by the same module grouping already used in the backend package structure (MDB Part-1 Section 6) — module-per-schema separation is not adopted, since it would complicate the cross-module Foreign Key relationships already catalogued in Appendix-H Section 5.
- Primary/replica topology (Appendix-L Section 9-10) is realized as a PostgreSQL streaming-replication primary plus at least one read replica; reporting-heavy queries (Reports module, Appendix-K Section 18) are directed to the replica via a routing datasource configuration at the Spring Boot layer.
- Partitioning of the high-volume tables already identified in Appendix-H Section 17.8 (AttendanceRecord, AuditLog, NotificationLog, Invoice/Payment) is implemented using native PostgreSQL declarative partitioning, keyed by academic_session_id or a time column as already specified per entity in Appendix-H.
- Connection pooling (HikariCP, the Spring Boot default) is mandatory for every Application Layer instance; pool sizing is tuned against the concurrent-user and connection-count targets in Appendix-F's Scalability NFR series, exact values remaining Client/Product Decision Required.

22. Entity Design Standards

Fixes JPA/Hibernate mapping conventions for realizing the 39 logical entities already fully specified in Appendix-H — this section governs how those entities are mapped, not what they contain.

Aspect	Standard
Primary Key	Single-column surrogate BIGINT, generated via a database sequence strategy — matches Appendix-H Section 2.3 exactly, no natural/composite keys as entity identifiers
Optimistic Locking	Every entity carries a JPA @Version-mapped integer field, realizing the version field already defined in Appendix-H Section 2.8
Soft Delete	Every entity carries an is_deleted flag plus deleted_by/deleted_at; queries exclude soft-deleted rows by default via a shared base-entity/filter mechanism, not repeated per-repository logic
Audit Columns	created_by/created_at/updated_by/updated_at populated automatically via Spring Data JPA Auditing, not set manually in Service code
Relationships	Mapped exactly per Appendix-H Section 5's Relationship Catalogue — cardinality, mandatory/optional, and cascade behavior (Restrict-only, never Cascade Delete, per Appendix-H Section 11) match precisely
Polymorphic Associations	Owner_type/owner_ref_id pattern (User, Document, ApprovalRequest, NotificationLog, BookIssue) implemented with an application-layer integrity check in the Service layer, since JPA cannot enforce a polymorphic Foreign Key constraint natively — directly addressing the risk flagged in Appendix-H Section 19
JSON Fields	Entity attributes already modeled as JSON in Appendix-H (salary_structure_json, deductions_json, grade_band_json, stops_json) are mapped via a JPA JSON type/converter, not decomposed into additional columns

23. Repository Layer Standards

Fixes Spring Data JPA as the concrete realization of the Repository component already defined in Appendix-J Section 5.

- Exactly one Spring Data JPA repository interface per entity, extending the standard repository base interface — no repository spans multiple entities, and no entity lacks a repository.
- A Repository contains only data-access method declarations (derived query methods or explicit query annotations) — no business logic, no validation, and no cross-repository orchestration, preserving the strict layer boundary already established in Appendix-J Section 6.
- Every list/search-style repository method that can return a high-volume result set (per Appendix-H Section 17.7 volume estimates) accepts a pagination parameter and returns a paginated result — an unpaginated "return everything" method is not permitted for these entities, consistent with Appendix-K Section 9.
- Custom queries needed beyond derived query methods use the framework's structured query mechanism (not native/raw SQL strings), preserving the SQL Injection Protection standard already fixed in MDB Part-2 Section 19.
- A Repository never calls another Repository or Service — if cross-entity data is needed, that composition happens in the Service layer, not the Repository layer.

24. Service Layer Standards

Fixes the concrete responsibilities of the Service component already defined in Appendix-J Sections 5-6, now bound to Spring Boot's service-bean conventions.

- Every Business Rule from Appendix-C v1.1 is enforced in exactly one Service method — no rule is duplicated across multiple Service methods, and no rule is left to the Controller, Repository, or frontend alone.
- A Service method depends only on Repository interfaces (its own module's, or another module's published Service interface per MDB Part-1 Section 8) — it never depends on another module's Repository or entity directly.
- Every Service method that changes state calls AuditLogService before returning (MDB Part-2 Section 20) and, where applicable, raises the appropriate domain event for cross-module effects (Fee posting from Library/Transport, notification triggers), consistent with the Event Publisher/Consumer pattern in Appendix-J Section 5.
- DTO-to-entity and entity-to-DTO mapping occurs at the Service layer boundary — a Controller never performs this mapping, and a Repository never returns a DTO.

25. Transaction Management Standards

Fixes Spring's declarative transaction management as the mechanism for the transactional consistency principle already established in Appendix-H Section 2.9 and Appendix-I Section 19.2.

- The transaction boundary is the Service method, not the Controller or Repository — a Service method implementing a multi-step Business Rule (e.g., BR-ADM-001/BR-ADM-007 seat allocation, BR-FEE-006 duplicate-payment prevention) executes as a single atomic transaction, fully committing or fully rolling back.
- The BR-ADM-001/BR-ADM-007 seat-allocation race condition, already flagged as an unresolved risk in Appendix-H Section 19 and Appendix-I Section 26.2, is mitigated using database-level row locking (pessimistic lock) or an equivalent atomic increment/compare-and-set at the SeatAllocation repository call, not merely optimistic-locking retry alone, given the safety-critical nature of the capacity ceiling.
- Transaction rollback is automatic on any Business Rule Exception or Concurrency Exception (MDB Part-2 Section 17); it is not automatic on a Validation Exception, since validation failures are expected to be caught before a transaction begins wherever possible.
- Read-only Service methods (Reports module, Search operations) are explicitly marked read-only at the transaction-annotation level, allowing the underlying connection pool and, where configured, the read-replica routing (Section 21) to optimize accordingly.
- No transaction spans an external integration call (Payment Gateway, SMS/Email Gateway) — the local database transaction commits or rolls back independently of the external call's outcome, with reconciliation handled via the Event Publisher/retry pattern already defined in Appendix-J Section 17, not a distributed transaction.

26. Caching Strategy

Fixes Redis as the concrete realization of the Cache Layer already defined in Appendix-I Section 17 and Appendix-F NFR-CACHE-001.

Aspect	Standard
Cache-Eligible Data	Slow-changing master/reference data only: Class, Section, Subject, GradingScheme, published TimetableEntry, Configuration entries — matching the candidates already identified in Appendix-I Section 17 and Appendix-L Section 6.7
Cache-Ineligible Data	Sensitive/PII fields (Appendix-G/H Sensitive Data Matrix) are never cached in Redis in plaintext; high-write-frequency transactional data (AttendanceRecord, Payment, MarksRecord) is not cached, since staleness risk outweighs read-performance benefit
Cache Pattern	Cache-aside (read-through on miss, explicit invalidation on write) — the database, not the cache, remains the system of record at all times
Invalidation Trigger	A Service-layer write to a cache-eligible entity explicitly evicts or updates the corresponding Redis key in the same logical operation, never left to time-based expiry alone for correctness-sensitive data
Session/Token Data	Refresh token state (MDB Part-2 Section 13) is a distinct Redis usage from application data caching, with its own eviction/expiry policy tied to token lifecycle, not the general cache-aside pattern
Distributed Consistency	Since the Application Layer scales horizontally (Appendix-I Section 18.1), Redis is the single shared cache instance/cluster across all instances — no per-instance local cache is used for data requiring cross-instance consistency

27. Performance Standards

Fixes concrete engineering practices for meeting the Performance NFR targets already defined in Appendix-F and reaffirmed in MDB Part-1 Section 4.

- Every JPA relationship defaults to lazy fetch; eager fetching is used only where a specific, documented access pattern requires it, avoiding uncontrolled N+1 query patterns across the high-fan-out Student and Employee entities (Appendix-D Critical Dependency List).
- Every index already recommended per entity in Appendix-H Section 12 is created as a corresponding database index at schema-creation time (Section 28) — index creation is not deferred to a later performance-tuning pass for entities already known to be high-volume.
- Batch operations (Bulk Marks Upload, Bulk Fee Collection, Bulk Attendance Entry — per the respective UI/UX Screen Specifications) use JPA/Hibernate batch-insert/update configuration rather than row-by-row persistence calls in a loop.
- Query result sets for any endpoint backing a Table Columns list (per the UI/UX Screen Specifications' Default Pagination field) are paginated at the database query level, not fetched in full and paginated in application memory.
- Response time budgets (Appendix-F NFR-PERF series, MDB Part-2 Section 15) are treated as a Code Review gate (Section 30) for any new Repository or Service method touching a high-volume entity, not verified only at the end-to-end Non-Functional Testing stage (Appendix-M Section 10).

28. Database Migration Standards

Governs how the PostgreSQL schema evolves over the project lifecycle, independent of the specific migration tool selected.

- Every schema change is expressed as a versioned, forward-only migration script, applied in strict sequence — a schema is never changed by direct, ad hoc DDL execution against any environment, including Development (Appendix-L Section 4).
- Migrations are reviewed under the same Code Review Checklist (Section 30) as application code, with specific attention to whether the change matches the logical model already approved in Appendix-H — a migration introducing a column, table, or constraint not traceable to Appendix-H or a formally logged Schema Extension item (per the UI/UX Screen Specifications' flagging convention) is rejected.
- Rollback/down-migrations are required for every schema change during Development and Testing environments; Staging and Production rollback follows the Rollback Strategy already defined in Appendix-L Section 21, favoring roll-forward fixes over destructive schema rollback where feasible.
- The specific migration tool (e.g., a Flyway/Liquibase-class tool) is not fixed in the Technology Stack provided to this MDB and remains Client/Product Decision Required; whichever tool is selected must support the versioned, forward-only, reviewable convention above without exception.

29. Development Best Practices

- Every feature branch is named with its Screen ID, FR ID, or BR ID (e.g., a branch implementing SCR-FEE-005 references that ID in its name), extending the commit-message traceability rule already fixed in MDB Part-1 Section 10.3 to branch-level granularity.
- No code is merged to the main integration branch without at least one independent review approval, consistent with the Independent Verification principle already established in Appendix-M Section 5.
- Every public Service and Repository method carries a documentation comment (Javadoc or equivalent) stating which FR/BR/NFR it implements — this is the code-level continuation of the traceability chain running from RGD v2.0 through every prior document in this set.
- Feature work is decomposed along the same module boundaries already fixed in MDB Part-1 Sections 6-8 — a single unit of work does not span multiple modules' packages unless it is specifically implementing a documented cross-module event (Section 24).
- Dead code, commented-out code blocks, and unused imports are not permitted to merge — the codebase is expected to reflect only the current, approved specification set at all times.

30. Code Review Checklist

Every pull request is checked against this list before approval, in addition to the language-specific Coding Standards already fixed in MDB Part-1 Section 10.

#	Checklist Item
1	Change traces to a specific FR, BR, NFR, Entity ID, or Screen ID from the approved documentation set
2	Layer boundaries respected: Controller has no business logic; Repository has no business logic; Service performs all Business Rule enforcement (Sections 23-24)
3	AuditLogService called for every state-changing Service method (MDB Part-2 Section 20)
4	Validation implemented at the correct tier per MDB Part-2 Section 16 (Bean Validation vs. Service-layer Business Rule check)
5	Transaction boundaries correct per Section 25 — no partial-commit risk on a multi-step operation
6	New/changed high-volume queries are paginated and use an existing or newly-justified index (Section 27)
7	Cache invalidation added for any write to a cache-eligible entity (Section 26)
8	Security Standards followed (MDB Part-2 Section 19) — no raw SQL, no plaintext Sensitive/PII logging, correct authorization annotation present
9	Unit tests present and passing for any new/changed Business-Rule-implementing Service method (Appendix-M Section 8)
10	Naming conventions followed exactly (MDB Part-1 Section 9) across Java, TypeScript, and any schema-affecting change
11	No dead code, no commented-out blocks, no unused imports/dependencies introduced